

UNIVERSIDADE ESTADUAL DE CAMPINAS
INSTITUTO DE COMPUTAÇÃO

MC714 — Sistemas Distribuídos

2º Semestre de 2026

Trabalho 1

Balanceador de Carga para um Sistema Distribuído

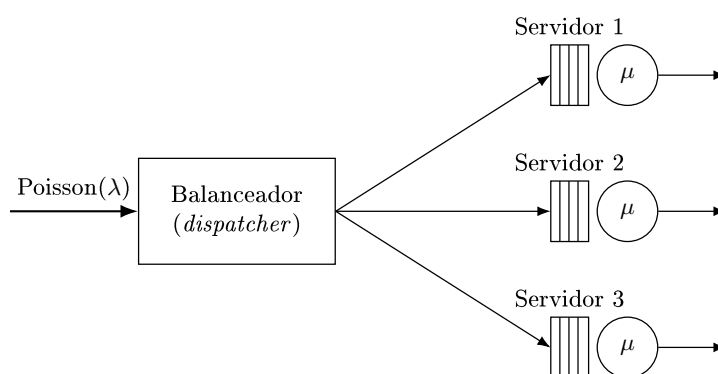
Carlos Alberto Astudillo Trujillo (Professor)
Diogo Maciel da Cunha (Estudante de Mestrado — PED)

Campinas, Agosto de 2026

1 Instruções

O balanceamento de carga em um sistema distribuído é uma técnica fundamental para otimizar o uso de recursos e garantir eficiência. O objetivo é distribuir a carga entre pontos do sistema, evitando que alguns fiquem sobrecarregados enquanto outros permanecem subutilizados. Este projeto visa a implementação e a análise de um balanceador de carga que receba requisições e as aloque em três servidores, levando em consideração políticas fundamentais, **modelagem analítica** e avaliação por simulação.

Entregue um relatório de até 4 páginas descrevendo o processo, além do código-fonte. Este projeto deverá ser desenvolvido em dupla. A avaliação levará em consideração o relatório e o código-fonte da solução.



1.1 Trabalho a fazer

Linguagem de programação e plataforma. Escolha qualquer linguagem de programação (por exemplo, Python, Java, C++, Go).

Modelo do sistema. Implemente um simulador do sistema da figura acima, com os seguintes parâmetros:

- **3 servidores homogêneos**, cada um com **uma única thread** de execução e uma fila FCFS (*First Come, First Served*) **ilimitada**. Ou seja: cada servidor atende **uma** requisição por vez; as demais esperam na sua fila.
- **Chegadas:** processo de **Poisson** de taxa λ — ou equivalentemente, tempos entre chegadas **exponenciais** de média $1/\lambda$.
- **Tempo de serviço: exponencial** de média $1/\mu$, com $\mu = 1,0$ requisição por unidade de tempo (isto é, $E[S] = 1$ u.t.).
- O balanceador não tem tempo de processamento próprio (é instantâneo).

Políticas de balanceamento. Implemente as três políticas a seguir; o programa deve ter uma opção de configuração para alternar entre elas:

- **Escolha Aleatória:** o balanceador sorteia um dos 3 servidores com probabilidade $1/3$ cada, de forma independente para cada requisição.
- **Round Robin:** o balanceador distribui as requisições de forma cíclica (1, 2, 3, 1, 2, 3, ...).
- **Fila Mais Curta:** o balanceador consulta o número de requisições em cada servidor (na fila + em execução) e envia para o menor; empates são resolvidos aleatoriamente.

Experimentos.

- Execute o sistema para $\lambda \in \{0,6; 1,2; 1,8; 2,4; 2,7\}$ requisições por unidade de tempo, para **cada** uma das três políticas (15 configurações no total).
- Cada execução deve ter duração de **5000 unidades de tempo**, descartando as **500 primeiras** unidades como período de aquecimento (*warm-up*).
- Cada configuração deve ser executada **10 vezes** com sementes aleatórias diferentes. Reporte a **média** e o **intervalo de confiança de 95 %** das 10 réplicas.
- **Métricas:** vazão do sistema (X), tempo médio de resposta ($E[R]$), número médio de requisições no sistema ($E[N]$) e utilização de cada servidor (U_i).
- Execute também **um** experimento adicional com $\lambda = 3,3$ e observe o que acontece com o tamanho das filas ao longo do tempo.

Modelagem analítica. Esta é a parte central do trabalho.

O modelo analítico deve ser desenvolvido **apenas para a política de escolha aleatória**. As políticas Round-Robin e Fila Mais Curta são avaliadas **somente por simulação** e comparadas com esse modelo, conforme o item (d).

- Mostre, usando o **teorema da decomposição** (*splitting*) de um processo de Poisson, que sob a política **aleatória** cada servidor recebe um processo de Poisson de taxa $\lambda_i = \lambda/3$, e que os três servidores são **independentes** entre si.
- Conclua que, sob a política aleatória, cada servidor é exatamente uma fila **M/M/1**. Deduza, a partir da distribuição estacionária $p_k = (1-\rho)\rho^k$, as expressões de U_i , $E[N_i]$, $E[T_Q]$ e $E[R]$. Determine a **condição de estabilidade** do sistema e a **vazão** X . Apresente uma tabela com esses valores para cada λ do experimento.
- Argumente que a **vazão** X e a **utilização** U_i previstas em (b) valem para as **três** políticas, e não apenas para a aleatória. (Dica: pense em conservação de trabalho — toda requisição que chega acaba sendo servida por algum servidor.) Confirme isso na sua simulação.
- Compare o $E[R]$ **simulado** das três políticas com o valor **analítico** $E[R] = \frac{1}{\mu - \lambda/3}$ obtido em (b). Verifique que

$$E[R]_{\text{sim}}^{\text{Fila curta}} \leq E[R]_{\text{sim}}^{\text{Round-Robin}} \leq E[R]_{\text{sim}}^{\text{Aleatória}} \approx \frac{1}{\mu - \lambda/3},$$

e **explique o porquê** desta ordenação. Quantifique o ganho percentual de cada política em relação à aleatória, para cada λ .

- Verifique a **Lei de Little** ($E[N] = X \cdot E[R]$) nos seus dados de simulação, para as três políticas.
- Para o caso $\lambda = 3,3$: explique por que o sistema é **instável** e por que as fórmulas dos itens anteriores deixam de valer. Use a aproximação fluida $N(t) \approx N(0) + (\lambda - 3\mu)t$ e compare com a curva de $N(t)$ obtida na simulação.

Comparação. Apresente uma tabela e um gráfico com, para cada valor de λ : a curva **analítica** de $E[R]$ (política aleatória) e os pontos **simulados** das **três** políticas, com barras de erro de 95 %. Discuta as diferenças observadas.

Detalhes da implementação.

- O balanceador deve possuir configuração para alternar entre as três políticas.
- A implementação deve registrar *logs* detalhados ou apresentar na saída padrão a dinâmica de distribuição de carga (quantas requisições foram para cada servidor, ocupação das filas ao longo do tempo).
- Comente o seu código adequadamente.

Documentação e relatório.

- Forneça instruções claras sobre como compilar e executar o seu código.
- Inclua a **dedução** e a **formulação** do modelo analítico, comparando seus resultados com os gráficos/dados da simulação.
- Inclua uma descrição da divisão de trabalho entre os membros da dupla (compatível com os *commits* do repositório).

Instruções de submissão. Submeta o seu código-fonte num único ficheiro comprimido (*.zip* ou *.tar.gz*). Certifique-se de que o código execute perfeitamente em ambiente Windows ou Linux.

Ponto extra (opcional). Escolha *um* dos itens:

- **Buffer finito:** limite a fila de cada servidor a K requisições ($K \in \{5, 10, 20\}$), descartando as excedentes. Modele cada servidor como $M/M/1/K$, calcule a probabilidade de perda $P_{\text{perda}} = p_K$ e a vazão efetiva $X = \lambda(1 - P_{\text{perda}})$, e compare com a simulação.
- **Servidores heterogêneos:** use $\mu_1 = 1,5$, $\mu_2 = 1,0$, $\mu_3 = 0,5$. Mostre que a divisão em $1/3$ deixa de ser ótima e proponha (e teste) pesos p_i proporcionais a μ_i .

1.2 Entregáveis

- `relatorio_projeto1_nome_de_um_integrante_do_grupo.pdf`
- `codigo_projeto1_nome_de_um_integrante_do_grupo.zip`

1.3 Data de entrega

A data de entrega é no dia 22 de setembro de 2026. Deverão subir os arquivos no Classroom dentro da tarefa do projeto. Apenas uma submissão por dupla é necessária.

2 Relatório

- Prepare um relatório de máximo 4 páginas no formato IEEE (coluna dupla).
- O relatório deve conter, pelo menos:
 - Resumo.
 - Descrição da arquitetura do balanceador e políticas adotadas.
 - Descrição dos servidores e parametrização do tráfego.
 - **Desenvolvimento da modelagem analítica** (itens (a)–(f) acima).
 - Resultados e discussão: comparação quantitativa entre os valores analíticos e a média das 10 execuções da simulação.
 - Conclusão.

3 Dicas

- Não comece o projeto dias antes da submissão. Planejem sua entrega e façam avanços oportunamente.
- É permitido utilizar bibliotecas de simulação de eventos discretos caso desejem (como SimPy em Python ou JSL em Java).
- **Cheque de sanidade:** com λ muito pequeno, $E[R]$ deve tender a $E[S] = 1$ u.t. para *qualquer* política. Se isso não acontecer, há um bug.
- Todo o modelo analítico deste trabalho sai de uma única fila **M/M/1**: se você estiver precisando de fórmulas mais complicadas, provavelmente errou o modelo.
- Reporte $E[N]$ medindo a *área sob a curva* do número de requisições no sistema (integral no tempo dividida pela duração), e não a média sobre chegadas.

4 Avaliação

O projeto abrange a avaliação do relatório e do código-fonte. O detalhe da ponderação é apresentado na Tabela 1.

- É proibido copiar o trabalho de outra dupla. Se detectado, todos os envolvidos serão penalizados.
- Critérios de avaliação:
 - **Corretude:** implementação correta das políticas, da simulação e da modelagem analítica.
 - **Validação:** consistência na comparação entre modelo analítico e simulação.
 - **Qualidade do código:** código modular, documentado e limpo.
 - **Documentação:** clareza e rigor técnico no relatório de 4 páginas.

Tabela 1: Avaliação do projeto.

Item	%
Relatório e Modelagem Analítica (itens (a)–(f))	35
Implementação da simulação (chegadas e servidores)	20
Implementação do Balanceador de Carga e das 3 políticas	20
Comparação quantitativa analítico $\times$ simulação	20
Qualidade do código e documentação	5